

**UNIVERSIDADE FEDERAL DE PELOTAS
CENTRO DE DESENVOLVIMENTO TECNOLÓGICO
CURSO DE CIÊNCIA DA COMPUTAÇÃO**

GUSTAVO SCHMITZ BERGMANN

**SOBREVIVÊNCIA JURÁSSICA:
IMPLEMENTAÇÃO DE UM JOGO ORIENTADO A OBJETOS COM INTERFACE
GRÁFICA EM JAVA**

Pelotas

2026

SUMÁRIO

1 Introdução.....	3
2 Diagrama de classes	4
3 Conceitos de orientação a objetos aplicados.....	6
3.1 Definição de classes em Java	6
3.2 Encapsulamento, construtores e métodos estáticos	6
3.3 Relacionamento entre classes: generalização, associação, agregação e composição	7
3.4 Herança.....	7
3.5 Polimorfismo.....	7
3.6 Classes abstratas e interfaces	8
3.7 Tratamento de exceções	8
3.8 Coleções	9
3.9 Interfaces gráficas	9
3.10 Multithreading.....	9
3.11 Arquivos e serialização.....	10
4 Compilação e execução do programa.....	10
5 Exemplos de utilização	11
6 Dificuldades encontradas.....	13
7 Sugestões de trabalhos para os próximos semestres.....	15
8 Conclusão	15

1 Introdução

Este relatório documenta o desenvolvimento do trabalho final da disciplina de Programação Orientada a Objetos, ministrada pelos professores Felipe Marques, no curso de Ciência da Computação da Universidade Federal de Pelotas (UFPel), referente ao período letivo 2026/1.

O trabalho consistiu na implementação do jogo “Sobrevivência Jurássica” na linguagem Java, com interface gráfica desenvolvida com a biblioteca Swing. No jogo, o usuário controla um personagem que deve explorar um mapa gerado aleatoriamente, coletar itens e armas, e sobreviver a encontros com quatro tipos de dinossauros — Compsognato, Troodonte, Velociraptor e Tiranossauro Rex — cada um com atributos, comportamento de movimentação e regras de combate próprios.

Diferentemente da etapa intermediária do trabalho, que utilizava apenas entrada e saída via console, esta versão final inclui uma interface gráfica completa, com telas de boas-vindas, escolha de dificuldade, tabuleiro interativo, janela de combate e tela de resultado, além de suporte a movimentação autônoma dos dinossauros em uma thread separada da interface e de um sistema de salvamento e carregamento de partidas em arquivo.

O restante deste documento está organizado da seguinte forma: o Capítulo 2 apresenta o diagrama de classes do sistema; o Capítulo 3 descreve onde e como cada conceito de orientação a objetos solicitado no trabalho foi empregado na implementação; o Capítulo 4 explica como compilar e executar o programa; o Capítulo 5 mostra telas reais de utilização do jogo; os Capítulos 6 e 7 discutem, respectivamente, as dificuldades encontradas durante o desenvolvimento e sugestões de continuidade para turmas futuras; e o Capítulo 8 apresenta a conclusão do trabalho.

2 Diagrama de classes

3 Conceitos de orientação a objetos aplicados

Esta seção indica, para cada conceito de orientação a objetos apresentado em aula e exigido pelo trabalho, onde e como ele foi efetivamente empregado na implementação do jogo.

3.1 Definição de classes em Java

Todo o sistema foi organizado em sete pacotes Java (Jogador_Mapa_Outros, Caixas_E_Itens, Dinossauros, Combate, Exceptions, Persistencia e GUI), somando cerca de trinta classes e interfaces. Cada classe foi definida com responsabilidade única e bem delimitada — por exemplo, a classe Jogador representa exclusivamente o personagem controlado pelo usuário (posição, saúde, percepção e inventário), enquanto a lógica de geração do mapa foi isolada na classe GeradorDeMapa, e a lógica de combate, na classe Combate.

3.2 Encapsulamento, construtores e métodos estáticos

Todos os atributos das classes de domínio são privados, sendo acessados apenas por meio de métodos públicos. Um exemplo representativo é a classe Personagem, cujos atributos saude, saudeMaxima, linha e coluna são privados e só podem ser alterados por métodos como receberDano(int) e setPosicao(int, int), que preservam invariantes (por exemplo, a saúde nunca fica negativa). A classe Inventario encapsula a lista de itens do jogador, expondo apenas operações de alto nível, como possuiLancaDardos() e getLancaDardos(); é o método Jogador.coletarItem(Item) quem decide, sem expor a estrutura interna do inventário, se um item novo deve ser adicionado ou apenas somado a um item já existente (caso da munição do Lança-Dardos).

Construtores customizados encadeados com super() aparecem em toda a hierarquia de Personagem: por exemplo, Dinossauro(int linha, int coluna, int saude) é chamado pelo construtor de cada subclasse concreta (Compsognato(int, int), Troodonte(int, int) etc.), que define apenas o valor de saúde inicial específico daquele tipo de dinossauro.

Métodos estáticos foram usados sempre que uma operação não depende do estado de uma instância específica. A classe GerenciadorPersistencia, por exemplo, é uma classe utilitária composta inteiramente por métodos estáticos

(salvar(EstadoJogo) e carregar()), sem necessidade de ser instanciada; e o método main(String[]), ponto de entrada do programa, está declarado como estático na classe JanelaPrincipal.

3.3 Relacionamento entre classes: generalização, associação, agregação e composição

Generalização (herança): a hierarquia Entidade → Personagem → {Jogador, Dinossauro} → {Compsognato, Troodonte, Velociraptor, TiranossauroRex} é o exemplo mais extenso de generalização do projeto; a hierarquia Item → Arma → {BastaoEletrico, LancaDardos} é outro exemplo, detalhado na seção 3.4.

Associação: a classe Combate associa-se a um Jogador, a um Dinossauro e a um Mapa apenas para a duração de um combate — ela utiliza esses objetos (chama métodos, lê e altera atributos), mas não é dona do ciclo de vida deles; os três continuam existindo normalmente depois que o objeto Combate é descartado ao final da luta.

Agregação: o Mapa mantém uma lista de Dinossauro (List<Dinossauro>) e uma referência ao Jogador posicionados sobre sua matriz, mas esses objetos não são criados nem destruídos pelo Mapa — eles são criados por GeradorDeMapa/JanelaPrincipal e apenas posicionados no tabuleiro, caracterizando uma relação de agregação (o “todo” referencia as “partes”, mas as partes têm vida própria).

Composição: o Jogador possui um Inventario criado em seu próprio construtor (new Inventario()) e que não existe fora do contexto de um jogador — relação de composição típica. Da mesma forma, cada CaixaDeSuprimentos é criada já contendo o seu ConteudoCaixa (o item ou o Compsognato-surpresa que ela guarda), e esse conteúdo não é compartilhado com nenhuma outra caixa.

3.4 Herança

O exemplo de herança mais diretamente ligado aos requisitos do trabalho é a hierarquia de armas: Item → Arma → {BastaoEletrico, LancaDardos}. A classe abstrata Arma define o método calcularDano(Random), e cada subclasse concreta o sobrescreve com sua própria regra de dado — o BastaoEletrico causa dano crítico (2 pontos) em resultado maior que 5, erra no resultado 1 e causa 1 ponto de dano nos

demais casos, enquanto o LancaDardos sempre causa dano crítico, mas consome uma munição por disparo. Da mesma forma, todas as quatro subclasses de Dinossauro herdam de uma superclasse abstrata comum, reaproveitando atributos (saúde, posição) e comportamentos genéricos definidos em Personagem e Dinossauro, e especializando apenas o que é próprio de cada espécie (dano de ataque, vulnerabilidades e padrão de movimentação).

3.5 Polimorfismo

O polimorfismo é utilizado de forma consistente para eliminar decisões condicionais (if/else encadeados) que, de outra forma, teriam que testar o tipo concreto de cada objeto. A thread de movimentação (ThreadDinossauros) percorre a lista de dinossauros do mapa chamando `dino.mover(mapa)` polimorficamente, sem nunca precisar saber se está lidando com um Troodonte (que persegue o jogador) ou um Velociraptor (que se move aleatoriamente por até duas casas) — cada subclasse decide sua própria direção de movimento. De forma semelhante, o Combate chama `arma.calcularDano(rand)` sem saber se a arma equipada é um BastaoEletrico ou um LancaDardos: cada uma resolve seu próprio cálculo de dano e devolve a mensagem correspondente por meio de `getUltimaMensagem()`.

3.6 Classes abstratas e interfaces

São classes abstratas do projeto: Entidade, Personagem, Dinossauro, Item e Arma — todas definem atributos e/ou métodos comuns às suas subclasses, mas nunca são instanciadas diretamente.

O projeto possui uma única interface, `ConteudoCaixa`, definida deliberadamente para representar “o que um objeto faz” (é encontrado dentro de uma caixa de suprimentos e reage a esse evento), e não “o que ele é”. Essa interface é implementada por duas famílias de classes sem nenhum ancestral comum: a classe Item (e, por herança, todas as suas subclasses — KitMedico, BastaoEletrico e LancaDardos) e a classe Compsognato, que pertence à hierarquia de Dinossauro. Um Compsognato pode, assim, aparecer tanto andando livremente pelo mapa quanto escondido dentro de uma caixa de suprimentos como uma “surpresa”, sem que a classe CaixaDeSuprimentos precise conhecer a diferença entre um item comum e um dinossauro.

3.7 Tratamento de exceções

Foram criadas três exceções de aplicação, todas checked (subclasses diretas de `Exception`), cada uma representando uma condição de erro previsível do domínio do jogo: `MovimentoInvalidoException`, lançada por `Mapa.moverJogador(int, int)` quando o destino é uma posição fora do tabuleiro, ocupada por uma parede, ou não adjacente à posição atual do jogador, e tratada em `PainelJogo` com uma mensagem amigável em `JOptionPane`; `SemMunicaoException`, lançada por `Combate` quando o jogador tenta atacar com o Lança-Dardos sem possuir a arma ou sem munição disponível; e `PersistenciaException`, que encapsula as exceções de mais baixo nível (`IOException`, `ClassNotFoundException`) lançadas pelo sistema de arquivos ou pela desserialização ao salvar ou carregar uma partida, traduzindo-as para uma mensagem específica do domínio da aplicação.

3.8 Coleções

O projeto utiliza a Java Collections Framework em diversos pontos: `Inventario` mantém os itens do jogador em uma `List<Item>` (implementada como `ArrayList`); `Mapa` mantém os dinossauros vivos em uma `List<Dinossauro>` e devolve as posições livres para fuga durante um combate em uma `List<int[]>` (`Mapa.posicoesParaFuga()`); a classe `Dinossauro` utiliza `Collections.shuffle(...)` para embaralhar a ordem das quatro direções possíveis de movimento a cada passo; e a interface gráfica (`PainelJogo`) mantém um cache de imagens já carregadas e redimensionadas em dois `HashMap` (`Map<Character, Image>` e `Map<String, ImageIcon>`), evitando reler e reprocessar os arquivos de imagem do disco a cada atualização do tabuleiro.

3.9 Interfaces gráficas

Toda a interface gráfica do jogo foi construída com a biblioteca Java Swing. A classe `JanelaPrincipal` (subclasse de `JFrame`) usa um `CardLayout` para alternar entre quatro telas independentes — `PainelBoasVindas`, `PainelDificuldade`, `PainelJogo` e `PainelResultado` — sem a necessidade de abrir novas janelas a cada troca de tela. O tabuleiro do jogo (`PainelJogo`) é implementado como uma grade de `JButton` organizados em um `GridLayout`, cada botão exibindo, por meio de um `ImageIcon`, a imagem do elemento presente naquela célula (jogador, parede, caixa ou dinossauro) e reagindo a cliques do jogador para movimentação. O combate é conduzido por uma

janela modal dedicada (CombateDialog, subclasse de JDialog), que bloqueia a interação com o tabuleiro até que a luta termine, e mensagens pontuais — coleta de itens, erros de movimento, confirmação de saída — são exibidas com JOptionPane.

3.10 Multithreading

A movimentação autônoma dos dinossauros é implementada em uma thread dedicada, a classe ThreadDinossauros (subclasse de Thread), que a cada ciclo percorre os dinossauros vivos do mapa chamando dino.mover(mapa) polimorficamente e, caso algum deles encontre o jogador, avisa a interface gráfica para iniciar um combate. Como o Swing não permite alterar componentes visuais fora da Event Dispatch Thread (EDT), toda atualização de tela originada por essa thread é despachada por meio de SwingUtilities.invokeLater(...). Para evitar condições de corrida entre essa thread e a própria interação do jogador (que também altera o tabuleiro, a partir da EDT), os métodos do Mapa que modificam a matriz do tabuleiro — tentarMoverEntidade(...), moverJogador(...), fugir(...), entre outros — são declarados synchronized; além disso, a thread pode ser pausada e retomada (pausar()/retomar()) sempre que um combate está em andamento, evitando que os demais dinossauros se movam durante uma luta.

3.11 Arquivos e serialização

O jogo permite salvar e carregar o progresso de uma partida em arquivo, utilizando serialização nativa de objetos Java. A classe EstadoJogo (pacote Persistencia) implementa Serializable e agrega o Mapa, o Jogador e o nível de percepção de uma partida; para isso, toda a cadeia de classes de domínio referenciada por ela — Mapa, Jogador, Dinossauro e suas subclasses, Item e suas subclasses, Inventario e CaixaDeSuprimentos — também implementa Serializable. A classe GerenciadorPersistencia grava esse estado em disco com um ObjectOutputStream e o recupera com um ObjectInputStream, encapsulando qualquer IOException ou ClassNotFoundException em uma PersistenciaException específica do domínio da aplicação, conforme descrito na seção 3.7. O mesmo mecanismo de serialização é reaproveitado para implementar a opção “Reiniciar Jogo”: o estado inicial de cada partida é clonado inteiramente em memória (serializando e imediatamente desserializando o próprio objeto EstadoJogo) logo após

a geração do mapa, permitindo restaurar exatamente a mesma configuração inicial sem precisar gerar um novo mapa aleatório.

4 Compilação e execução do programa

Esta seção descreve o ambiente utilizado para compilar e executar o programa, bem como o passo a passo necessário para reproduzi-lo.

4.1 Ambiente utilizado

- Sistema operacional: utilizei o Windows 11 para rodar e testar o programa.
- IDE sugerida: NetBeans, pois é a mesma utilizada em aula.
- Bibliotecas externas: nenhuma. O projeto utiliza somente as bibliotecas padrão do Java (javax.swing, java.awt, java.io, java.util).

4.2 Passo a passo (NetBeans)

1. Crie um novo projeto Java com fontes existentes (File → New Project → Java with Existing Sources) e aponte a pasta de código-fonte para o diretório src/.
2. Copie os arquivos de imagem para a raiz do projeto (mesmo nível da pasta src/), para que fiquem no diretório de trabalho padrão de execução do NetBeans.
3. Defina GUI.JanelaPrincipal como classe principal do projeto (clique com o botão direito no projeto → Properties → Run → Main Class).
4. Execute o projeto com Build Project e depois Run Project.

5 Exemplos de utilização

Esta seção apresenta capturas de tela reais, obtidas a partir da execução do programa compilado, ilustrando o fluxo típico de uma partida.

A Figura 2 mostra a tela de boas-vindas, primeira tela exibida ao iniciar o programa, com as opções “Jogar”, “Carregar Jogo” (habilitada apenas quando existe uma partida salva em disco) e “Sair”.



Figura 2 – Tela de boas-vindas

Fonte: os autores.

Ao selecionar “Jogar”, o usuário é encaminhado para a tela de escolha de dificuldade (Figura 3), que define o valor inicial do atributo percepção do jogador: 3 no modo Fácil, 2 no modo Médio e 1 no modo Difícil.



Figura 3 – Tela de escolha de dificuldade

Fonte: os autores.

Definida a dificuldade, o mapa é gerado aleatoriamente e a tela principal do jogo é exibida (Figura 4): o tabuleiro é mostrado como uma grade de botões clicáveis,

cada um com o ícone do elemento correspondente (jogador, parede, caixa de suprimentos ou dinossauro) dentro do campo de visão do jogador; a barra superior mostra a saúde e a percepção do personagem, a quantidade de dinossauros restantes, e os botões de ação “Usar Kit Médico”, “DEBUG (revela o mapa inteiro)”, “Salvar Jogo” e “Sair do Jogo”.

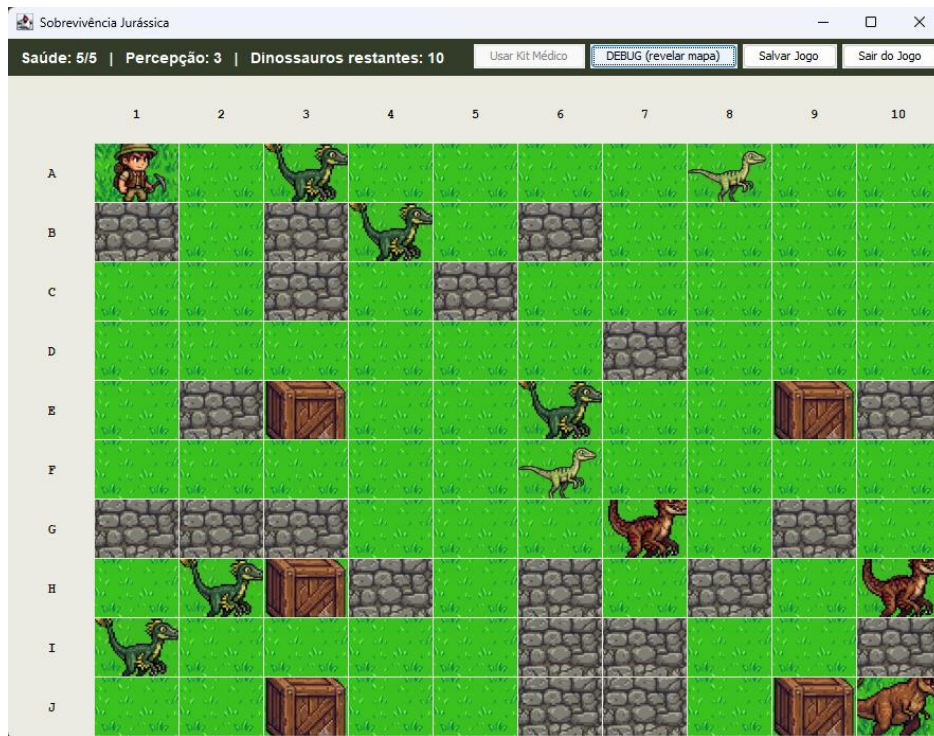


Figura 4 – Tela principal do jogo, com o tabuleiro já povoado

Fonte: os autores.

Ao mover o personagem para uma célula ocupada por um dinossauro — ou quando um dinossauro se move até uma posição adjacente ao jogador —, uma janela de combate modal é aberta, permitindo atacar com as mãos (ou com o Bastão Elétrico, se equipado), atacar com o Lança-Dardos (se equipado e com munição disponível) ou tentar fugir para uma posição adjacente livre, conforme as regras descritas na seção 3.4.

6 Dificuldades encontradas

Ao longo do desenvolvimento, diversas dificuldades técnicas foram encontradas e resolvidas. As principais estão resumidas a seguir.

- Excesso de abstração em iterações intermediárias: em determinado momento do desenvolvimento, o sistema de combate e de movimentação dos

dinossauros acumulou abstrações mais sofisticadas do que o necessário para o escopo da disciplina (uma thread independente por dinossauro, uma interface, um sistema de armas genérico baseado em categorias). Essas soluções funcionavam corretamente, mas dificultavam a explicação do código em uma apresentação. Foi necessário revisar deliberadamente a arquitetura para simplificá-la, preservando o comportamento do jogo, mas concentrando a movimentação dos dinossauros em uma única thread e voltando a um tratamento de combate mais direto.

- Sincronização entre a thread de movimentação e a interface gráfica: como o Swing não permite atualizar componentes visuais fora da Event Dispatch Thread (EDT), foi necessário garantir que toda alteração da interface originada pela thread de movimentação dos dinossauros fosse repassada por meio de `SwingUtilities.invokeLater(...)`, e que o tabuleiro (classe `Mapa`) protegesse com métodos `synchronized` qualquer operação que pudesse ser chamada tanto pela thread de movimentação quanto pela própria interação do usuário.
- Ícones do tabuleiro não preenchendo os botões corretamente: ao substituir a representação do tabuleiro (letras coloridas) por imagens, surgiram dois problemas em sequência. Primeiro, o tamanho dos componentes (`JBUTTON.getWidth()/getHeight()`) ainda não estava definido no momento em que os ícones eram redimensionados, pois o gerenciador de layout só define o tamanho real dos componentes depois que a janela é efetivamente exibida pela primeira vez — isso fazia os ícones serem redimensionados para 0x0 pixels. Depois de corrigido, um segundo problema surgiu: como o ícone tinha um tamanho fixo, mas o botão podia ser bem maior (dependendo do tamanho da janela), sobrava um espaço vazio ao redor da imagem. A solução final recalcula o ícone de cada célula sempre que a grade do tabuleiro muda de tamanho (usando um `ComponentListener`), mantendo um cache por símbolo e tamanho para não prejudicar o desempenho.
- Geração aleatória de um mapa sempre válido: a geração do tabuleiro precisa garantir posições livres suficientes para o jogador, o Tiranossauro Rex (posicionado na extremidade oposta ao jogador), os demais dinossauros e as caixas de suprimentos, mesmo em mapas com até 20% de paredes. Foi necessário incluir uma rotina de tentativas aleatórias com um mecanismo de

repescagem determinística (varredura linha a linha) para os casos raros em que o sorteio aleatório não encontra uma posição livre dentro de um número razoável de tentativas.

7 Sugestões de trabalhos para os próximos semestres

Uma possibilidade é o desenvolvimento de um jogo de exploração espacial, no qual o jogador percorre diferentes planetas coletando recursos, enfrentando alienígenas e aprimorando sua nave. Outro projeto interessante seria um jogo ambientado em um castelo medieval, no qual o personagem deve explorar salas, derrotar monstros, coletar equipamentos e resolver desafios para avançar na história. Também pode ser proposto um jogo de sobrevivência em uma ilha deserta, exigindo que o jogador administre recursos, construa ferramentas e enfrente eventos aleatórios para permanecer vivo.

Além de alterar o tema do jogo, futuras turmas podem ser desafiadas a implementar funcionalidades mais avançadas, como sistema de missões, árvore de habilidades, inventário com gerenciamento de peso, diferentes níveis de dificuldade, efeitos sonoros, animações, etc.

Essas propostas permitem que os estudantes consolidem os fundamentos da Programação Orientada a Objetos ao mesmo tempo em que entram em contato com técnicas de desenvolvimento de software mais avançadas. Dessa forma, os projetos permanecem motivadores, aproximam os alunos de situações reais de desenvolvimento e estimulam a criatividade, o trabalho em equipe e a aplicação prática dos conhecimentos adquiridos ao longo da disciplina.

8 Conclusão

O desenvolvimento do jogo Sobrevivência Jurássica permitiu aplicar, em um único projeto de porte considerável, praticamente todos os conceitos de orientação a objetos trabalhados ao longo da disciplina: definição de classes, encapsulamento, construtores e métodos estáticos, os quatro tipos de relacionamento entre classes (generalização, associação, agregação e composição), herança, polimorfismo, classes abstratas, interfaces, tratamento de exceções e uso de coleções.

Mais do que simplesmente implementar cada conceito isoladamente, o trabalho exigiu decidir, a cada etapa, qual conceito melhor resolvia um problema real de design — por exemplo, reconhecer que a movimentação de cada tipo de dinossauro era um caso natural de polimorfismo, ou que uma caixa de suprimentos podia conter tanto um item quanto um dinossauro-surpresa por meio de uma interface comum. Esse processo também envolveu, em mais de um momento, reconhecer quando uma solução havia se tornado mais sofisticada do que o problema exigia, e simplificá-la sem perder as propriedades de orientação a objetos que a disciplina buscava desenvolver.

Por fim, a adição de uma interface gráfica completa, de uma thread dedicada à movimentação dos dinossauros e de um sistema de salvamento em arquivo demonstrou, na prática, como os mesmos princípios de encapsulamento e responsabilidade única aplicados às classes de domínio do jogo também se estendem à camada de interação com o usuário e à infraestrutura de persistência, resultando em um sistema passível de manutenção e extensões futuras.